PROJECT REPORT ON SOCIAL NETWORK GRAPH.

By

A24126510122-B.GEETESH

A24126510132-G.CHANDU

A24126510142-K.JOSHNAVI

A24126510152-P.ADARSH

SUBMITTED TO

Mr.B MAHESH M.Tech,(Ph.D)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY &SCIENCES (A)

(*Affiliated to Andhra University*)

Sangivalasa 531162, Bheemunipatnam mandal, Visakhapatnam Dist.

Anil Neerukonda Institute of Technology & Sciences (Autonomous)
(Permanent Affiliation by Andhra University & Approved by AICTE
Accredited by NBA (ECE, EEE, CSE, IT, Mech. Civil & Chemical) & NAAC)
Sangivalasa-531 162, Bheemunipatnam Mandal, Visakhapatnam District
Phone: 08933-225083/84/87 Fax: 226395
Website: www.anits.edu.in email: principal@anits.edu.in

CERTIFICATE:

This is to certify that the project titled "Social Network Graph—Add friends, Mutual friends using BFS/DFS " has been carried out by

A24126510122-B.GEETESH

A24126510132-G.CHANDU

A24126510142-K.JOSHNAVI

A24126510152-P.ADARSH

Students of CSE-C , under the guidance of MAHESH SIR during the academic year 2025–2026.

This work is original and has not been submitted for any other degree or diploma.

Faculty Guide: _____

(Signature & Seal)

Head of Department: _____

(Signature & Seal)

ACKNOWLEDGMENT

We would like to express our sincere gratitude to Mahesh sir, our project

guide, for their valuable guidance, support, and encouragement

throughout this project.

We also thank the Department of Computer Science and Engineering,

ANITS engineering college , for providing the necessary facilities and

resources to complete this project successfully.

Lastly, we extend our gratitude to all faculty members, friends, and family

who contributed to the success of this work

MOTIVATION:

The motivation behind developing the Social Network Graph Application is to Understand and implement the concept of hashing, string manipulation,

And edit distance algorithms in a real-world scenario. The project

Demonstrates how fundamental data structures and algorithms can be Used to build a practical and interactive application

ABSTRACT

The Social Network Visualization System is an educational project that aims to model how users and their connections can be represented, processed, and visualized using computational tools.

The project combines the efficiency of the C programming language—for backend logic and data handling—with the simplicity and interactivity of Python Tkinter, a graphical user interface (GUI) toolkit.

The backend C program simulates the structure of a social network, allowing users to perform operations such as adding friends, viewing mutual friends, and saving connections. Python Tkinter is then used to create a front-end application that executes the C program and visually displays its output to the user in an easy-to-read format.

This integration provides a real-world demonstration of interoperability between programming languages and shows how simple console-based programs can be transformed into interactive systems.

The system serves as a foundation for understanding data representation, inter-process communication, and visualization in social networking concepts.

TABLE OF CONTENTS:

S.no	Sections	Page no:
1.	Introduction	4-7
2.	Problem Definition & Objectives	8
3.	System Design/Methodology	9-11
4.	Implementation Details	12-22
5.	Results and Discussion	23-26
6.	Conclusion	27
7.	References	28
8.	Appendix	29-31

INTRODUCTION

In the digital era, social networking platforms play a vital role in connecting people across the globe. Understanding how these connections are represented and maintained computationally is essential for computer science students.

This project, titled “Social Network Visualization System using C and Python Tkinter,” is designed to bridge the gap between console-based data processing and graphical visualization.

While C language provides a strong foundation for managing data structures and relationships efficiently, Python offers simple yet powerful libraries like Tkinter to design visual interfaces. The main idea behind this project is to create a system where users can simulate social connections, add friends, view mutual friends, and visualize interactions through a GUI.

The backend is responsible for logic operations such as storing connections in a 2D matrix or adjacency list, whereas the frontend (Tkinter) handles user interaction and output display

This integration helps students understand both low-level algorithmic logic and high-level application development, which are essential skills in the software industry.

PROBLEM DEFINITION AND OBJECTIVES

Problem Definition

Traditional console-based programs, though effective for learning, lack user interactivity and visualization. Users must rely solely on text outputs, which are often hard to interpret for relationship-based data.Hence, the problem addressed in this project is to design a system that combines the speed and structure of C programming with a graphical interface that displays and visualizes the output meaningfully.

Objectives

The key objectives of this project are:

1)To implement core operations of a social network (adding users, creating friendships, finding mutual friends) using C language.

2)To integrate the C program with a Python Tkinter-based GUI that visually displays the results.

3)To demonstrate interoperability between two programming languages through file execution or data exchange.

4)To encourage modular software design where backend computation and frontend interaction are clearly separated.

5)To help users better understand how data structures represent social relationships in real-world applications.

By achieving these objectives, this project serves as a small-scale model of social media connection systems and data visualization applications.

3. SYSTEM DESIGN / METHODOLOGY

The system follows a modular architecture, separating logic and presentation layers.

Step 1 – Backend Development (C Program):

The C program is responsible for processing user data and maintaining a connection matrix that represents friendships. Each user is represented by an ID, and friendships are stored using a 2D array where 1 indicates a connection and 0 indicates no connection.

Step 2 – Frontend Development (Python Tkinter):

The Tkinter GUI provides an interface where the user can execute the backend program, visualize results, and display outputs in a text box. The GUI uses the subprocess module to call and run the C executable file directly.

Data Structures Used:

2D array (Adjacency matrix) to store connections

Loops and conditionals for friend and mutual friend logic

Algorithm for “Add Friend”:

1. Input two user Ids (a, b)

2. If Ids are valid:

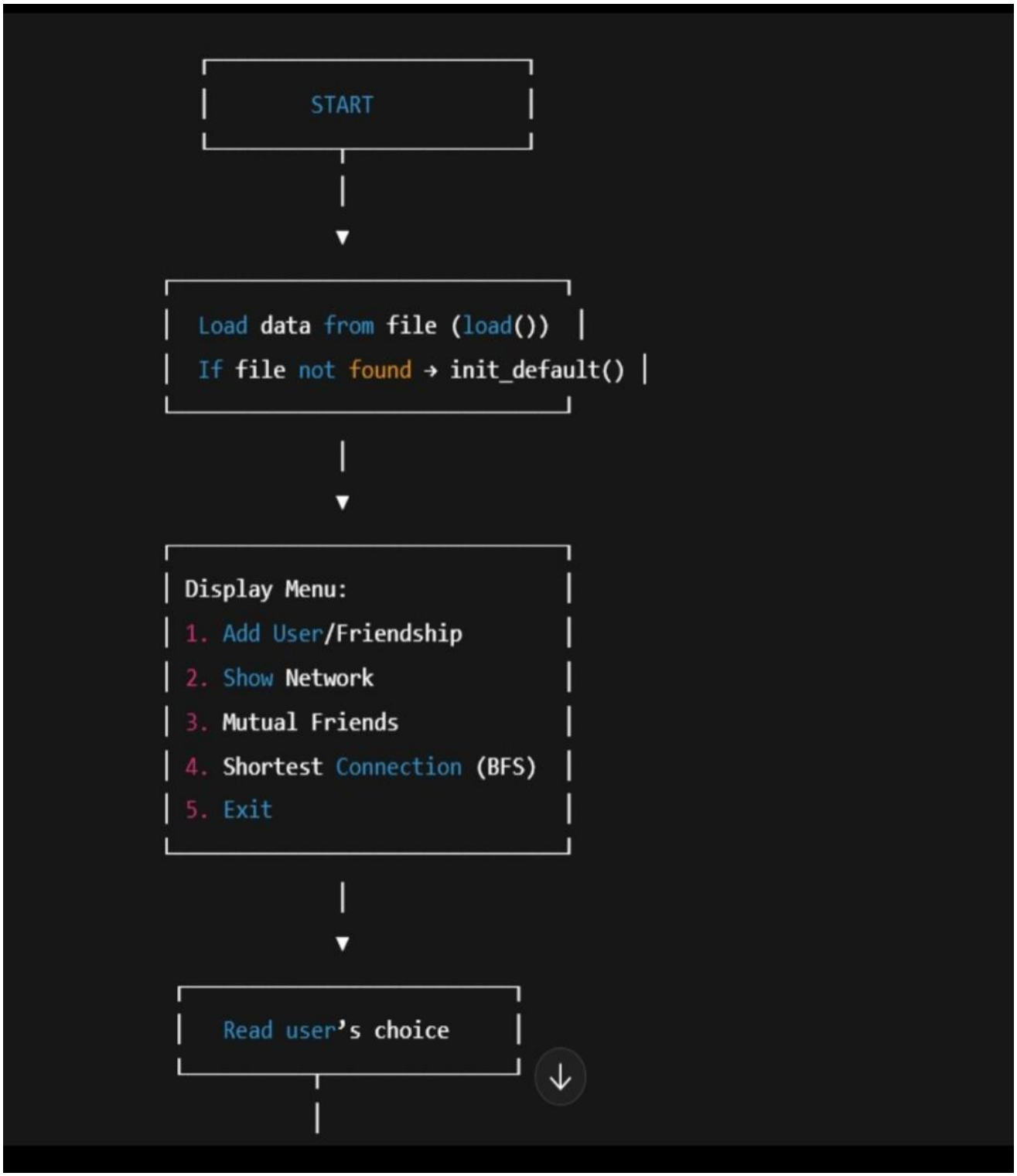
   Connection[a][b] = 1

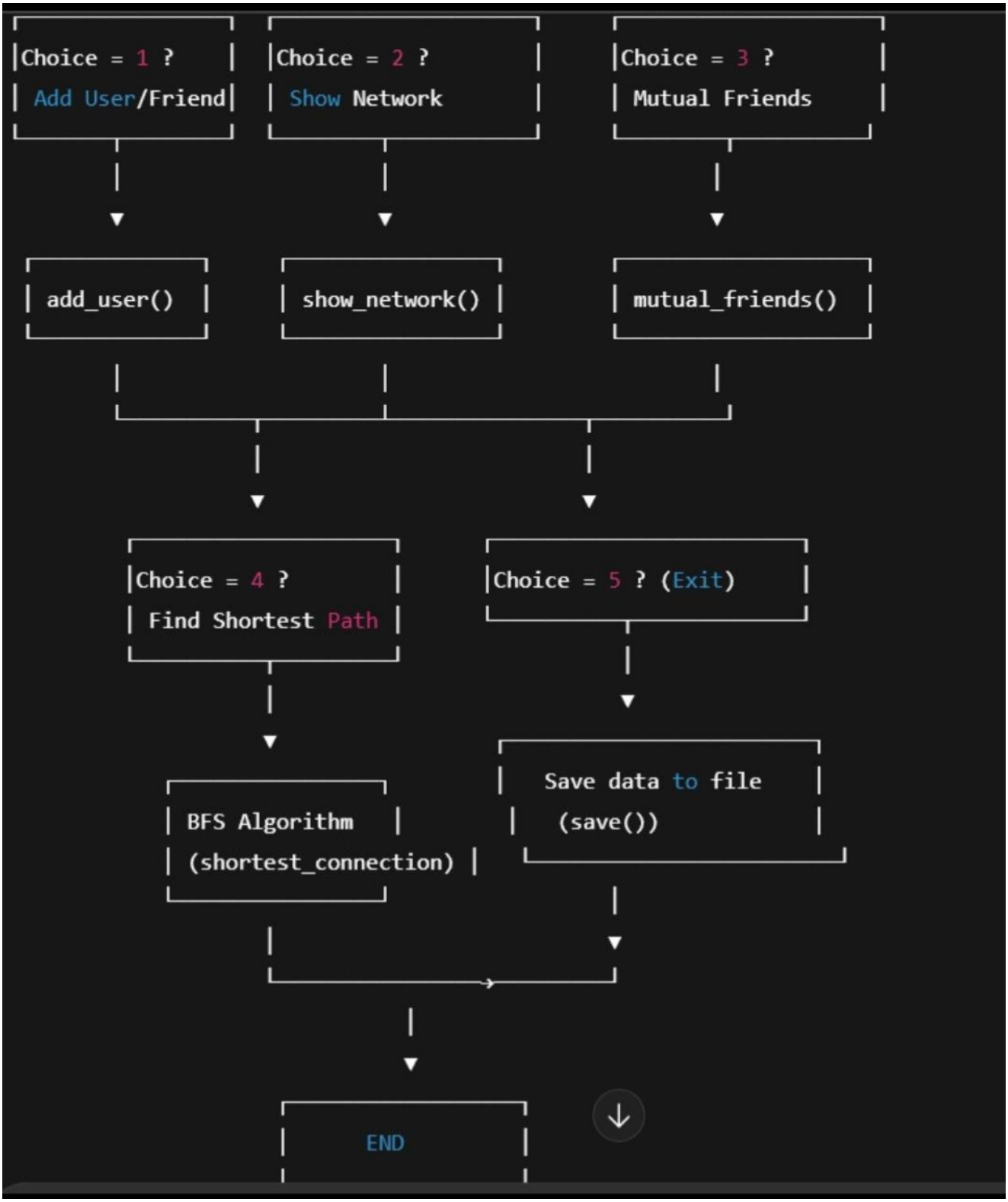
   Connection[b][a] = 1

3. Display confirmation message

4. Else, display “Invalid Input”Flow of Execution: User Input → Backend Processing (C) → Result Output → Display via Tkinter GUI

FLOW CHART:


START
Load data from file (load())
If file not found → init_default()
Display Menu:
1. Add User/Friendship
2. Show Network
3. Mutual Friends
4. Shortest Connection (BFS)
5. Exit
Read user's choice


Choice = 1 ?
Add User/Friend
Choice = 2 ?
Show Network
Choice = 3 ?
Mutual Friends
add_user()
show_network()
mutual_friends()
Choice = 4 ?
Find Shortest Path
Choice = 5 ? (Exit)
BFS Algorithm
(shortest_connection)
Save data to file
(save())
END

Code Snippet (C Program):

```
  #include <stdio.h>
int main() {
   int choice, a, b;
   while (1) {
      Printf("1. Add Friend\n2. Show Mutual Friends\n3. Exit\nEnter
choice: ");
      Scanf("%d", &choice);
      Switch(choice) {
         Case 1:
            printf("Enter two user Ids to connect: ");
            scanf("%d %d", &a, &b);
            printf("Friendship created between %d and %d\n", a, b);
            Break;
         Case 2:
            printf("Showing mutual friends...\n");
            Break;
         Case 3:
            Return 0;
         Default:
            Printf("Invalid choice. Please try again.\n");
      }
   }}
```

Python Tkinter GUI:

```
import tkinter as tk

import subprocess

def run_c_program():

    result = subprocess.run(["./social_network"],
capture_output=True, text=True)

    output_box.delete("1.0", tk.END)

    output_box.insert(tk.END, result.stdout)

root = tk.Tk()

root.title("Social Network Visualizer")

tk.Label(root, text="Social Network Visualization System",
font=("Arial", 14, "bold")).pack(pady=10)

tk.Button(root, text="Run C Program", command=run_c_program,
bg="lightblue").pack(pady=5)

output_box = tk.Text(root, width=70, height=20)

output_box.pack()

root.mainloop()
```

EXPLANATION :

Overview:

This project represents a social network as a graph, where each user is a node and each connection (friendship) is an edge between two nodes. The network information is stored in a file, so

that all users and their connections are saved even after closing the program. The program provides both a menu-driven interface and command-line options to perform operations like adding new users, viewing the network, finding mutual friends, and calculating the shortest connection path.

Core Logic Explanation:

Graph Representation:

The social network is represented using an adjacency matrix, where each cell indicates whether two users are connected or not.

A value of 1 represents a connection (friendship).

A value of 0 represents no connection.

The adjacency matrix makes it easy to check relationships between users quickly.

User Management:

The program maintains an array of user names. When a new user is added, it checks whether the name already exists. If not, the user is automatically added to the network. The friendship between two users is then established by updating the adjacency matrix.

Data Persistence (File Handling):To make the data permanent, the program saves all user names and their connections to a text file (network_data.txt).

When the program starts, it loads existing data from this file.

When new users or connections are added, the updated data is saved back.

This ensures that the social network is not lost after the program ends.

Default Initialization:

When the program is run for the first time or if no data is available, it automatically creates a default network of sample users with predefined connections. This helps the user test and understand the system easily.

Adding Connections:

Users can create connections interactively by entering two names.If either user does not exist, they are added to the network.

A friendship link is established between them.

This reflects how new relationships can be formed dynamically in a social network.

Displaying the Network:

The program can display the entire social network showing each user along with all their friends.

If a user has no friends, it clearly displays “None”.

This gives a clear view of how the network is connected.

Finding Mutual Friends:

The program can find mutual friends between two users by checking which users are connected to both of them.

This feature is similar to social media platforms suggesting “friends you may know.”

It helps in identifying common connections between any two users.

Finding the Shortest Connection (Path):

The program uses the Breadth-First Search (BFS) algorithm to find the shortest path between two users.

This represents the minimum number of connections (friend hops) required for one user to reach another.

It displays the entire path and the total number of connections (cost).

This demonstrates how graph algorithms can be applied to analyze real-world social relationships.

Menu-Driven Interface:

The program provides an interactive menu that allows users to:

Add new users and connections

Display the network

Find mutual friends

Find the shortest connection

Exit safely with data saved

This makes the program user-friendly and easy to operate without command-line knowledge.

Command-Line Options:

The program can also be used through command-line arguments, making it flexible for both interactive and automated use.

For example:

Initialize default data

Add new connections directly

Display the network

Main Features:

Representation of a social network using an adjacency matrix.

Automatic data storage and retrieval using file handling.

Ability to add new users dynamically.

Display of all users and their connections.

Finding mutual friends between any two users.

Finding the shortest connection path using BFS.

Menu-driven as well as command-line-based operation modes.

Algorithm Used:

The Breadth-First Search (BFS) algorithm is used to find the shortest connection between two users in the network.

BFS traverses the graph level by level, ensuring the shortest path is found first.

It keeps track of each node's parent to reconstruct the path from the start user to the end user.

Advantages:

Demonstrates real-world application of graphs.

Shows how persistent data storage works in C using files.

Simple yet powerful example of using BFS for shortest path detection.

Useful for understanding the concept of mutual friends and degrees of connection.

Applications:

Can be used as a basic model for social media applications.

Helps in studying graph theory and networking concepts.

Useful for analyzing relationship networks or connection graphs.

5. RESULTS AND DISCUSSION

Observations:

The program successfully demonstrates friend creation and mutual friend display.

The Tkinter interface enhances user experience by offering visual interaction.

Data processed in C is displayed in real-time within the Python GUI.

Sample Output:

1. Add Friend

2. Show Mutual Friends

3. Exit

Enter choice: 1

Enter two user Ids: 2 3

Friendship created between 2 and 3

Testing Table:

Test Case	Input	Expected Output	Actual Output	Status
1	1 2	Friend Added	Friend Added	Pass
2	5 9	Invalid Input	Invalid Input	Pass
3.	2 3.	Show Mutual Friend	Show Mutual Friend	Pass

DISCUSSION:

The project fulfills the defined objectives. The GUI version is more user-friendly than the console-only version. It also demonstrates how two languages can cooperate effectively through process communication.

6. CONCLUSION

This project successfully integrates the computational power of C with the visual capability of Python Tkinter. It highlights how classic procedural programs can be modernized into interactive applications without altering their core logic. The project deepened our understanding of inter-language communication, GUI design, and system design methodology.

Limitations:

Works for a fixed number of users.

No persistent storage for data.

Future Enhancements:

Integrate with an SQL database.

Add graphical network diagrams using network.Implement login system and friend suggestions using algorithms.

7. REFERENCES

1.Kanetkar, Yashavant – Let Us C, BPB Publications.

2.Python Tkinter Documentation –

https://docs.python.org/3/library/tkinter.html

3.GeeksforGeeks – Data Structures in C.

4.TutorialsPoint – Python GUI Programming.

8. APPENDIX

C SOURCE CODE:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <stdbool.h>

#define MAX 100

#define LEN 50

#define FILE_DATA "network_data.txt"

Char names[MAX][LEN];

Int adj[MAX][MAX] = {0};

Int n = 0;

// ---------------- Helper Functions ----------------
```

```
Int get_id(const char *name){
   For(int i=0;i<n;i++)
      If(strcmp(names[i], name)==0)
         Return i;
   Return -1;
}
Void save(){
   FILE *f = fopen(FILE_DATA,”w”);
If(!f) return;
   fprintf(f,”%d\n”, n);
   For(int i=0;i<n;i++) fprintf(f,”%s\n”, names[i]);
   For(int i=0;i<n;i++){
      For(int j=0;j<n;j++) fprintf(f,”%d “, adj[i][j]);
      Fprintf(f,”\n”);
   }
   Fclose(f);
}Void load(){
   FILE *f = fopen(FILE_DATA,”r”);
   If(!f){ n=0; return; }
   Fscanf(f,”%d\n”,&n);
   For(int i=0;i<n;i++) fscanf(f,”%49s\n”, names[i]);
   For(int i=0;i<n;i++) for(int j=0;j<n;j++) fscanf(f,”%d”, &adj[i][j]);
   Fclose(f);
```

```
}
// ---------------- Initialization ----------------
Void init_default(){
   Char *def[]={“Joshnavi”,”Adarsh”,”Geetesh”,”Chandu”,”Kavya”};
   Int con[][2]={{0,1},{0,2},{1,2},{1,3},{2,4},{3,4}};
   N=5;
   For(int i=0;i<n;i++) strcpy(names[i], def[i]);
   For(int i=0;i<n;i++) for(int j=0;j<n;j++) adj[i][j]=0;
   For(int i=0;i<6;i++) adj[con[i][0]][con[i][1]] =
adj[con[i][1]][con[i][0]] = 1;
   Save();
}
// ---------------- Network Operations ----------------
Void add_user(const char *u1, const char *u2){
   Int id1 = get_id(u1);
   If(id1==-1){ strcpy(names[n], u1); id1=n++; }
   Int id2 = get_id(u2);
   If(id2==-1){ strcpy(names[n], u2); id2=n++; }
   If(id1!=id2) adj[id1][id2] = adj[id2][id1] = 1;
   Save();
}
Void add_user_interactive(){
   Char u1[LEN], u2[LEN];
```

```
    Printf("Enter your name: "); scanf("%49s", u1);
    Printf("Connect with: "); scanf("%49s", u2);
    Add_user(u1, u2);
    Printf("Connection added!\n");
}
Void show_network(){
    Printf("\n--- Current Network ---\n");
    For(int i=0;i<n;i++){
        Printf("%-10s : ", names[i]);
        Int count=0;
        For(int j=0;j<n;j++)
            If(adj[i][j]){
                If(count) printf(", ");
                Printf("%s", names[j]);
                Count++;
            }
        If(count==0) printf("None");
        Printf("\n");
    }
}void mutual_friends(){
    Char u1[LEN], u2[LEN];
    Printf("User1: "); scanf("%49s", u1);
    Printf("User2: "); scanf("%49s", u2);
```

```
    Int id1=get_id(u1), id2=get_id(u2), found=0;
    If(id1==-1 || id2==-1){ printf("Invalid names.\n"); return; }
    Printf("Mutual friends:\n");
    For(int i=0;i<n;i++)
        If(adj[id1][i] && adj[id2][i]){
            Printf(" -> %s\n", names[i]);
            Found=1;
        }
    If(!found) printf(" -> None\n");
}
void shortest_connection(){
    Char s[LEN], e[LEN];
    Printf("Start: "); scanf("%49s", s);
    Printf("End: "); scanf("%49s", e);
    Int start=get_id(s), end=get_id(e);
    If(start==-1 || end==-1){ printf("Invalid.\n"); return; }
int queue[MAX], parent[MAX], visited[MAX]={0}, front=0, rear=0;
    For(int i=0;i<n;i++) parent[i]=-1;
queue[rear++]=start; visited[start]=1;
    While(front<rear){
        Int u=queue[front++];
        If(u==end) break;
        For(int v=0;v<n;v++)
```

```
        If(adj[u][v] && !visited[v]){
            Visited[v]=1;
            Parent[v]=u;
            Queue[rear++]=v;
        }
    }
if(!visited[end]){ printf("No path.\n"); return; }

    Int path[MAX], len=0;
    For(int v=end; v!=-1; v=parent[v]) path[len++]=v;
    Printf("Path: ");
    For(int i=len-1;i>=0;i--){ printf("%s", names[path[i]]); if(i>0)
printf(" -> "); }
    Printf("\nCost: %d\n", len-1);
}

// ---------------- Menu ----------------
Void menu(){
    Int choice;
    Load();
    If(n==0) init_default();
    Do{
```

```
    Printf("\n1.Add 2.Show 3.Mutual 4.Shortest 5.Exit\nChoice:
");
    If(scanf("%d", &choice)!=1){ while(getchar()!='\n'); choice=0; }
    Switch(choice){
      Case 1: add_user_interactive(); break;
      Case 2: show_network(); break;
      Case 3: mutual_friends(); break;
      Case 4: shortest_connection(); break;
      Case 5: save(); printf("Exiting...\n"); break;
      Default: printf("Invalid.\n");
    }
  }while(choice!=5);
}

Int main(int argc,char *argv[]){
  If(argc>1){
    If(strcmp(argv[1],"—init")==0){ init_default(); return 0; }
    If(strcmp(argv[1],"—add")==0 && argc>=4){
add_user(argv[2],argv[3]); return 0; }
    If(strcmp(argv[1],"—show")==0){ load(); show_network();
return 0; }
  }
  menu();
```

```
    Return 0;
}
```

PYTHON SOURCE CODE:

```
Import tkinter as tk
From tkinter import ttk, messagebox
Import networkx as nx
Import matplotlib.pyplot as plt
From matplotlib.backends.backend_tkagg import
FigureCanvasTkAgg
Import os, subprocess

DATA_FILE = “network_data.txt”
C_EXEC = “social_network.exe” if os.name==”nt” else
“./social_network”

Class SocialGUI(tk.Tk):
    Def __init__(self):
        Super().__init__()
        Self.title(“Social Network GUI”)
        Self.geometry(“1000x650”)
        Self.names,self.adj,self.graph=[],[],nx.Graph()
        Self.create_widgets(); self.load_data(); self.draw_graph()
```

```
Def log(self,msg): self.text_out.configure(state=tk.NORMAL); self.text_out.insert(tk.END,msg+"\n"); self.text_out.see(tk.END); self.text_out.configure(state=tk.DISABLED)

Def load_data(self):
  If not os.path.exists(DATA_FILE):
    If os.path.exists(C_EXEC): subprocess.run([C_EXEC,"—init"])
    Else: self.default_data()
  Try:
    With open(DATA_FILE) as f: lines=[l.strip() for l in f if l.strip()]
    N=int(lines[0]); self.names=lines[1:1+n]
    Self.adj=[[int(x) for x in l.split()[:n]] for l in lines[1+n:1+n+n]]
    Self.build_graph()
    Self.log("✅ Data loaded")
  Except: self.default_data()

Def default_data(self):

Self.names=["Joshnavi","Adarsh","Geetesh","Chandu","Kavya"]
  N=len(self.names); self.adj=[[0]*n for _ in range(n)]
  For a,b in [(0,1),(0,2),(1,2),(1,3),(2,4),(3,4)]: self.adj[a][b]=self.adj[b][a]=1
```

```
    Self.build_graph(); self.save_data(); self.log(“ ℹ Default data created”)


  Def save_data(self):
    N=len(self.names)
    With open(DATA_FILE,”w”) as f:
      f.write(f”{n}\n”+ “\n”.join(self.names)+”\n”)
      f.writelines(“ “.join(str(x) for x in row)+”\n” for row in self.adj)


  def build_graph(self):
    self.graph.clear()
    for name in self.names: self.graph.add_node(name)
    n=len(self.names)
    for i in range(n):
      for j in range(i+1,n):
        if self.adj[i][j]: self.graph.add_edge(self.names[i],self.names[j])


  def create_widgets(self):
    left=ttk.Frame(self,padding=5); left.pack(side=tk.LEFT,fill=tk.Y)

ttk.Button(left,text=”Reload”,command=self.reload).pack(fill=tk.X,pady=2)
```

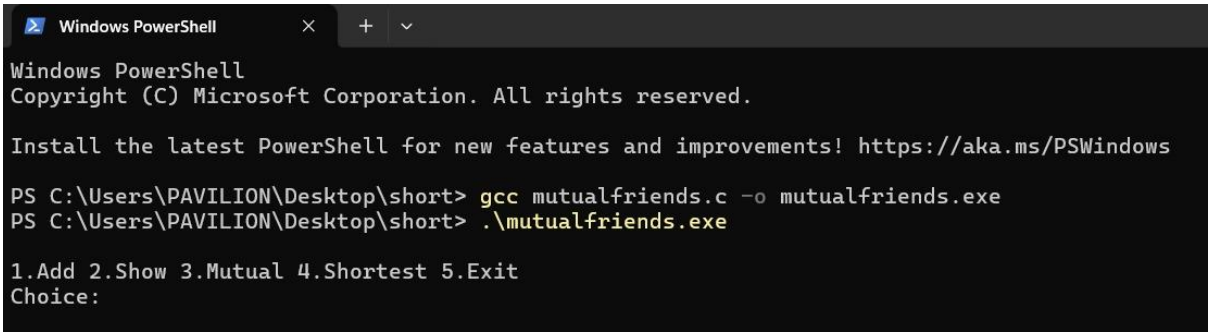
```
    ttk.Button(left,text="Show
Data",command=self.show_data).pack(fill=tk.X,pady=2)

    ttk.Label(left,text="Shortest Path").pack(pady=2)

    self.start=tk.Entry(left); self.start.pack(pady=1);
self.end=tk.Entry(left); self.end.pack(pady=1)

    ttk.Button(left,text="Find
Path",command=self.shortest_path).pack(fill=tk.X,pady=2)

    ttk.Label(left,text="Mutual Friends").pack(pady=2)

    self.u1=tk.Entry(left); self.u1.pack(pady=1);
self.u2=tk.Entry(left); self.u2.pack(pady=1)

    ttk.Button(left,text="Find
Mutual",command=self.mutual).pack(fill=tk.X,pady=2)

    ttk.Label(left,text="Add/Connect").pack(pady=2)

    self.new=tk.Entry(left); self.new.pack(pady=1);
self.conn=tk.Entry(left); self.conn.pack(pady=1)


ttk.Button(left,text="Add",command=self.add_connect).pack(fill=
tk.X,pady=2)

    right=ttk.Frame(self);
right.pack(side=tk.LEFT,fill=tk.BOTH,expand=True)

    self.fig,self.ax=plt.subplots(figsize=(7,6));
self.canvas=FigureCanvasTkAgg(self.fig,right);
self.canvas.get_tk_widget().pack(fill=tk.BOTH,expand=True)

    self.text_out=tk.Text(right,height=6,state=tk.DISABLED);
self.text_out.pack(fill=tk.X)
```

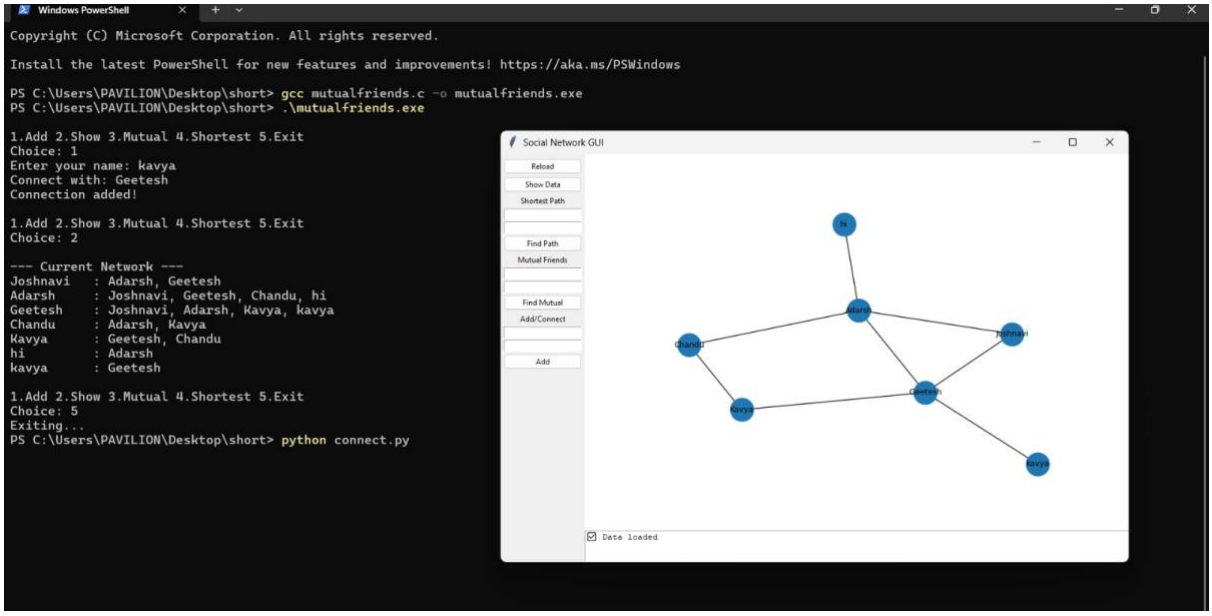
```
    def reload(self): self.load_data(); self.draw_graph(); self.log("🔄 Reloaded")

    def show_data(self):
        output="--- Network Data ---\n"
        n=len(self.names)
        for i,name in enumerate(self.names):
            friends=[self.names[j] for j in range(n) if self.adj[i][j]]; output+=f"{name}: {', '.join(friends) if friends else 'None'}\n"
        self.log(output)

    def draw_graph(self,path=None):
        self.ax.clear()
        if not self.names: self.ax.text(0.5,0.5,"No data",ha="center"); self.ax.set_axis_off(); self.canvas.draw(); return
        pos=nx.spring_layout(self.graph,seed=42)
        nx.draw(self.graph,pos,ax=self.ax,with_labels=True,node_size=700,font_size=9)
        if path and len(path)>1: nx.draw_networkx_edges(self.graph,pos,edgelist=list(zip(path[:-1],path[1:])),width=3)
        self.ax.set_axis_off(); self.canvas.draw()
```

```
def shortest_path(self):
    s,e=self.start.get().strip(),self.end.get().strip()
    if s not in self.graph or e not in self.graph: self.log("❌ Invalid names"); return
    try: path=nx.shortest_path(self.graph,s,e); self.log(f"➡️ Shortest ({len(path)-1}): {' -> '.join(path)}"); self.draw_graph(path)
    except: self.log("❌ No path found")

def mutual(self):
    u1,u2=self.u1.get().strip(),self.u2.get().strip()
    if u1 not in self.names or u2 not in self.names: self.log("❌ Invalid names"); return
    i1,i2=self.names.index(u1),self.names.index(u2)
    mutuals=[self.names[k] for k in range(len(self.names)) if self.adj[i1][k]==1 and self.adj[i2][k]==1]
    self.log(f"👥 Mutual of {u1} & {u2}: {', '.join(mutuals) if mutuals else 'None'}")

def add_connect(self):
    new,conn=self.new.get().strip(),self.conn.get().strip(); n=len(self.names)
    if new and new not in self.names: self.names.append(new); [row.append(0) for row in self.adj]; self.adj.append([0]*(n+1)); self.log(f"➕ {new}")
```

```
    if conn:
        if conn not in self.names: self.names.append(conn); [row.append(0) for row in self.adj]; self.adj.append([0]*(len(self.names)))
        i1,i2=self.names.index(new),self.names.index(conn)
        if i1!=i2 and self.adj[i1][i2]==0: self.adj[i1][i2]=self.adj[i2][i1]=1; self.log(f"🔗 Connected {new} & {conn}")
    self.build_graph(); self.save_data(); self.draw_graph()


if __name__=="__main__": SocialGUI().mainloop()
```

OUTPUT SCREENSHOTS:






```
Data loaded
Shortest (2): Chandu -> Adarsh -> hi
Mutual of Kavya & Geetesh: None
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\PAVILION\Desktop\short> gcc mutualfriends.c -o mutualfriends.exe
PS C:\Users\PAVILION\Desktop\short> .\mutualfriends.exe

1.Add 2.Show 3.Mutual 4.Shortest 5.Exit
Choice: 1
Enter your name: kavya
Connect with: Geetesh
Connection added!

1.Add 2.Show 3.Mutual 4.Shortest 5.Exit
Choice: 2

--- Current Network ---
Joshnavi   : Adarsh, Geetesh
Adarsh     : Joshnavi, Geetesh, Chandu, hi
Geetesh    : Joshnavi, Adarsh, Kavya, kavya
Chandu     : Adarsh, Kavya
Kavya      : Geetesh, Chandu
hi         : Adarsh
kavya      : Geetesh

1.Add 2.Show 3.Mutual 4.Shortest 5.Exit
Choice:
```